

RELATÓRIO DE ANÁLISE DE EFICIÊNCIA: Algoritmos de Resolução de Sudoku

Instituto Metr pole Digital - Universidade Federal do Rio Grande do Norte

Bacharelado em Tecnologia da Informa  o

Disciplina: Estruturas de Dados B sicas II

Autores: Gabriel Ferreira Cavalcante, Jordson Albino Galv o Tavares, Lyan Kelvin Ben cio Silva, Vin cius Maranh o Bahia de Ara jo

Reposit rio: https://github.com/GabsFerrarii/atividade_edb2

1. Introdu  o

O Sudoku   um problema de satisfa  o de restri  es no qual os valores de cada linha, coluna e bloco devem obedecer a regras de unicidade. Embora sua forma tradicional utilize um tabuleiro 9 9, o problema pode ser generalizado para diferentes dimens es. Do ponto de vista computacional, a resolu  o do Sudoku generalizado   NP-completa, o que torna relevante o estudo de estrat gias capazes de reduzir o espa o de busca e de como essas estrat gias se comportam   medida que o tamanho da entrada cresce.

Entre as diferentes abordagens de resolu  o encontram-se m todos de busca e m todos baseados em regras. Berggren e Nilsson (2012) analisam, entre outros solucionadores, o Backtracking e o Rule-Based. O primeiro explora recursivamente possibilidades de preenchimento e retorna quando uma escolha n o conduz a uma solu  o. O segundo procura reduzir previamente o espa o de busca por meio de regras l gicas e recorre a guessing e backtracking quando essas regras n o s o suficientes.

Este trabalho realiza uma an lise emp rica desses dois algoritmos em Sudokus generalizados. O objetivo   comparar o tempo de execu  o e a quantidade de opera  es realizadas conforme o tamanho da entrada aumenta, relacionando o comportamento observado   an lise assint tica. Para isso, foram utilizados tabuleiros de diferentes bases, v rias inst ncias por tamanho e repeti  es de cada execu  o, mantendo o ambiente de teste constante e aproximadamente a mesma propor  o de c lulas inicialmente vazias.

2. Algoritmos Avaliados e Explora  o do Espa o de Busca

Os dois algoritmos resolvem o mesmo problema, mas percorrem o espa o de solu  es de maneiras diferentes. O *Backtracking* explora candidatos recursivamente, enquanto o *Rule-Based* tenta primeiro deduzir o espa o de busca por meio de regras l gicas.

2.1. Algoritmo de *Backtracking* (Busca Exaustiva)

O *Backtracking* resolve o Sudoku por busca recursiva. Em cada estado, o algoritmo procura uma célula vazia com o menor número de candidatos possíveis, estratégia conhecida como *MRV (Minimum Remaining Values)*. Em seguida, testa os candidatos dessa célula e chama o solucionador recursivamente. Se uma escolha produzir um estado inválido ou não levar à solução, o algoritmo retorna ao estado anterior e tenta outro candidato.

Essa escolha por menor número de candidatos segue a estratégia descrita por Berggren e Nilsson (2012) e reduz o fator de ramificação da busca em comparação com a seleção da primeira célula vazia. Ainda assim, no pior caso o método pode explorar um grande número de combinações e apresenta crescimento exponencial/combinatorial.

2.2. Algoritmo *Rule-Based* (Baseado em Regras)

O segundo método aplica regras lógicas antes de recorrer à busca. A primeira é *naked single*: quando uma célula vazia possui apenas um candidato possível, esse valor pode ser atribuído diretamente. A segunda é *naked tuple*: quando um conjunto de células de uma mesma região compartilha exatamente um conjunto restrito de candidatos, esses candidatos podem ser eliminados das demais células daquela região.

As regras são reaplicadas enquanto produzem alterações. Quando nenhuma delas consegue avançar, o algoritmo escolhe uma célula com poucos candidatos e realiza um *guess*, usando recursão e *backtracking* de forma semelhante ao primeiro método. Assim, o *Rule-Based* combina redução lógica do espaço de busca com uma etapa de força bruta para garantir que a resolução possa prosseguir quando as regras não forem suficientes.

2.3. Adaptações dos algoritmos

A implementação foi adaptada a partir do código-fonte que consta no artigo original. O estudo de Berggren e Nilsson limita os testes ao Sudoku tradicional 9x9. Aqui, a estrutura do tabuleiro foi generalizada para diferentes bases. *Backtracking* e *Rule-Based* foram separados em classes distintas e foram adicionados contadores de operações, medição de tempo e suporte a limite de execução, sem alterar a lógica original dos algoritmos.

3. Metodologia

3.1. Entradas e definição de n

O parâmetro de construção utilizado é a base b do Sudoku. Para uma base b , cada bloco possui dimensão $b \times b$ e o lado completo do tabuleiro é b^2 . Neste trabalho, n foi definido como o número total de células do tabuleiro, de forma que $n = b^4$. A bateria final utilizou base 2, 3 e 4, correspondendo, respectivamente, a tabuleiros 4x4, 9x9, 16x16, com $n = 16, 81$ e 256 células.

Para cada tamanho, foram preparados três *puzzles* distintos. As entradas mantiveram aproximadamente a mesma proporção de células vazias, próxima de 50%, para evitar que o aumento do tamanho fosse acompanhado de uma mudança sistemática na quantidade relativa de pistas. Cada *puzzle* foi fornecido sem alteração aos dois algoritmos.

Para cada combinação de tamanho, *puzzle* e algoritmo, foram realizadas 10 repetições, sempre usando a mesma entrada para os dois métodos. O tempo de resolução foi medido com `std::chrono::steady_clock` e registrado em microssegundos e milissegundos. A leitura e a preparação das entradas ficaram fora do trecho cronometrado, de modo que a medição representasse apenas o custo da resolução.

Todos os testes foram executados em ambiente com C++17 e g++. A máquina utilizada para todas as execuções de teste foi um notebook Dell com Windows 11 Pro, Intel i7-1355U e 16GB de RAM.

3.2. Uso de Inteligência Artificial

O ChatGPT foi utilizado exclusivamente como apoio à adaptação, revisão e geração de parte do código e como auxílio ao entendimento dos conceitos utilizados no trabalho. Seu uso não foi empregado para a geração dos resultados experimentais.

4. Resultados e Discussão

4.1. Comparação dos tempos de execução

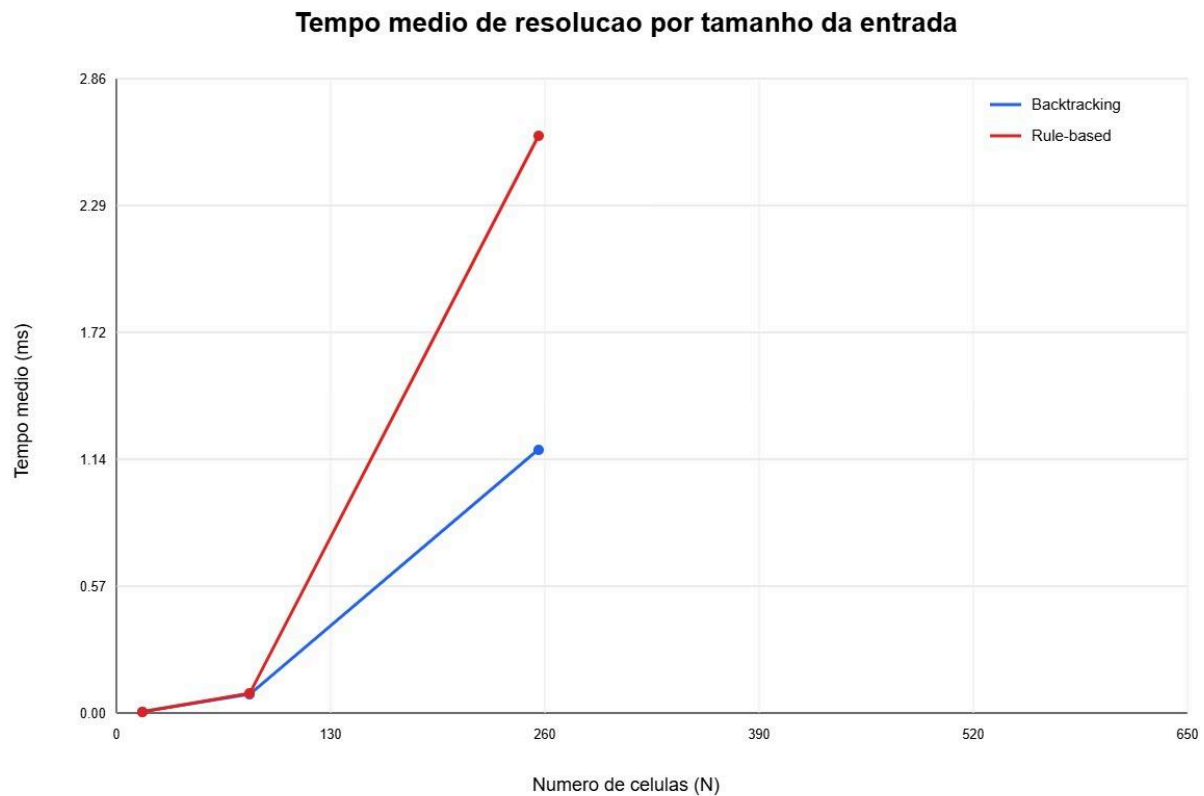
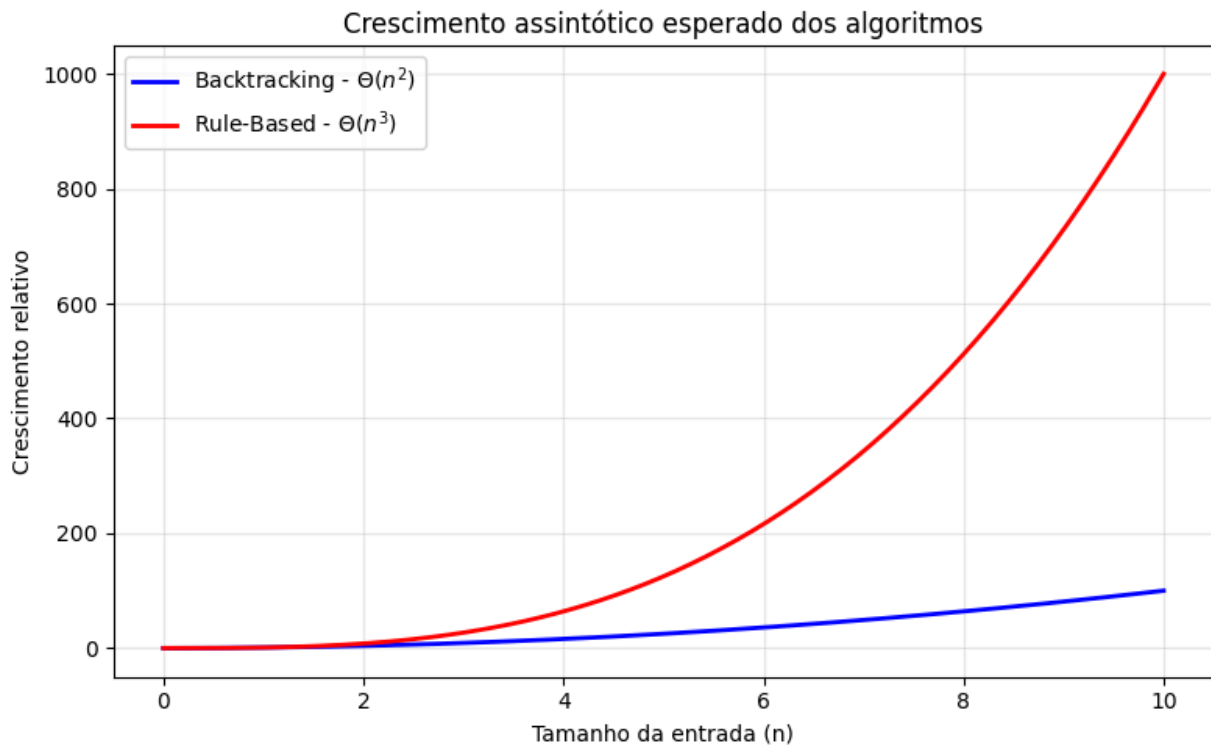
Tabuleiro	n	Backtracking (ms)	Rule-Based (ms)
4×4	16	0,006	0,006
9×9	81	0,085	0,089
16×16	256	1,189	2,61

O gráfico de tempo mostra crescimento do custo conforme n aumenta, mas não uma vantagem uniforme de um algoritmo em todas as instâncias. Nos tabuleiros 4×4, o *Rule-Based* geralmente resolveu os casos com poucas decisões de busca. Nos tabuleiros 9×9, houve puzzles em que o custo das regras tornou o *Rule-Based* mais lento e outros em que a redução da busca compensou esse custo. O mesmo comportamento ficou ainda mais claro no 16×16: em uma instância o *Rule-Based* foi mais rápido, enquanto nas demais o custo das regras superou a economia obtida com a redução de *guesses* e *backtracks*.

Esse resultado é compatível com a própria natureza dos métodos. O *Backtracking* possui uma operação de busca mais simples, mas pode explorar muitos ramos. O *Rule-Based* tenta reduzir esse espaço antes de recorrer à busca, pagando o custo adicional de examinar candidatos e combinações. Com apenas três tamanhos de entrada, não é adequado inferir uma classe de complexidade pelo formato da curva; os gráficos devem ser interpretados em conjunto com a análise teórica.

Para a comparação assintótica adotada neste trabalho, considera-se como crescimento esperado $\Theta(n^2)$ para o Backtracking e $\Theta(n^3)$ para o Rule-Based, tomando n como o número total de células do tabuleiro. O gráfico a seguir ilustra essas ordens de crescimento de forma

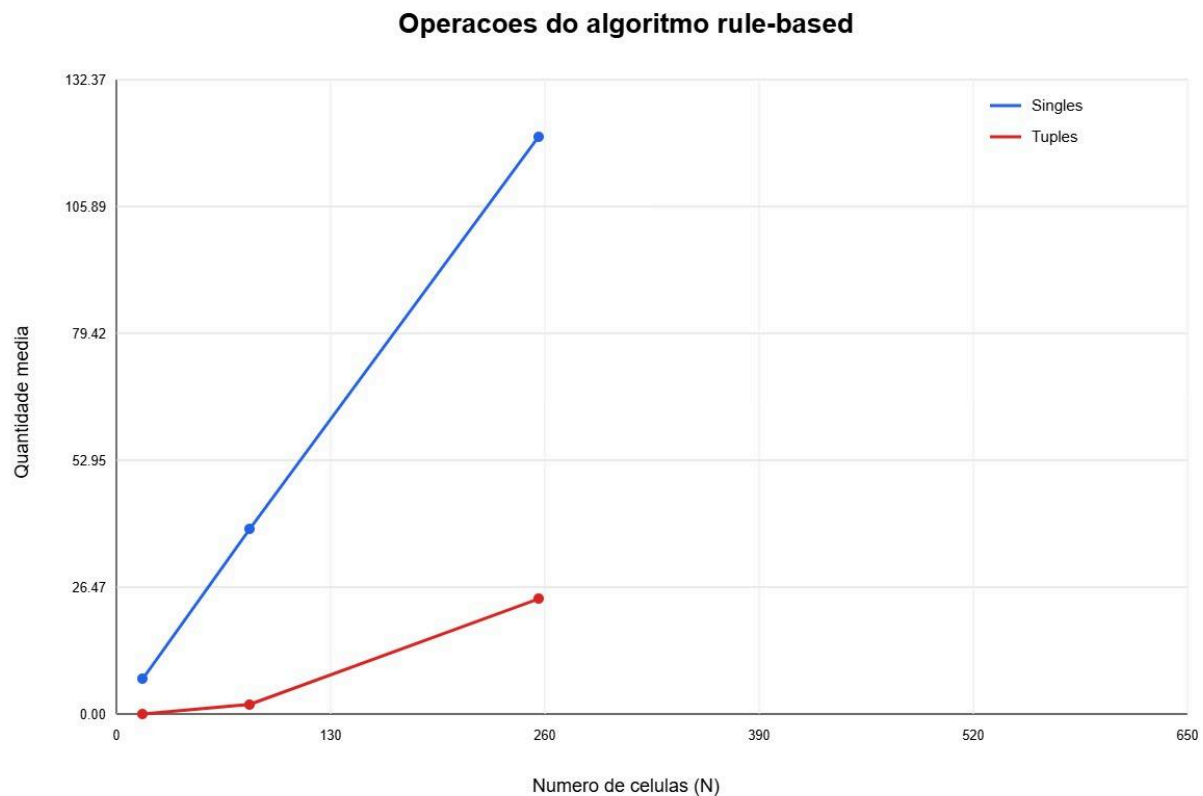
relativa e não representa tempos previstos de execução.

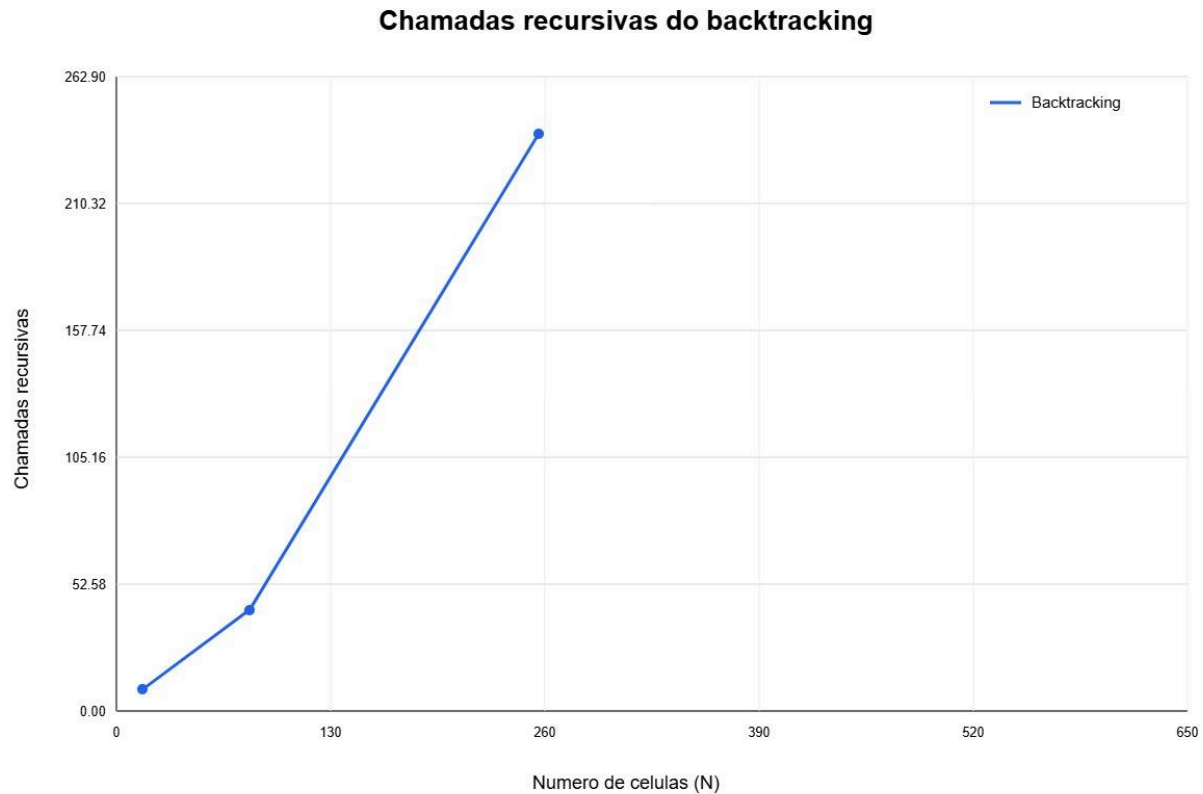


4.2. Comparação das operações realizadas

Os contadores internos ajudam a explicar os tempos observados. No *Backtracking*, as instâncias 16×16 exigiram 205, 164 e 348 chamadas recursivas nos três puzzles, com 85, 39 e 233 *backtracks*, respectivamente. Isso evidencia que a estrutura específica da entrada influencia bastante a árvore de busca.

No *Rule-Based*, as mesmas instâncias 16×16 foram resolvidas com apenas 8 a 10 chamadas recursivas e 7 a 9 guesses, sem *backtracking*. Em contrapartida, foram aplicados entre 119 e 121 *naked singles* e entre 7 e 34 *naked tuples*. O algoritmo, portanto, reduziu fortemente a busca, mas realizou trabalho adicional para localizar e aplicar regras. Esse custo é particularmente relevante na versão generalizada, pois a busca por *naked tuples* envolve combinações de células dentro de regiões cada vez maiores.





4.3. Limitações

A análise realizada possui limitações experimentais e limitações relacionadas aos próprios algoritmos. Embora os puzzles tenham sido preparados com aproximadamente a mesma proporção de células vazias, essa característica não garante dificuldade equivalente entre as instâncias. A quantidade de pistas, isoladamente, não determina a dificuldade de um Sudoku, pois a disposição dessas pistas e as deduções possibilitadas por elas também influenciam diretamente o processo de resolução. O uso de múltiplos puzzles por tamanho reduz esse efeito, mas não o elimina completamente.

Outra limitação decorre da generalização de algoritmos originalmente avaliados em Sudokus 9×9 para tabuleiros maiores. No Backtracking, a heurística MRV reduz o fator de ramificação em muitos casos, porém não elimina o pior caso combinatorial da busca. No Rule-Based, as regras podem diminuir consideravelmente a necessidade de guessing, mas possuem custo próprio. Em particular, a procura por naked tuples exige examinar combinações dentro de regiões que crescem junto com o tamanho do tabuleiro. Além disso, quando as regras não são suficientes, o algoritmo volta a utilizar guessing e backtracking, mantendo a possibilidade de crescimento combinatorial.

Os tempos também foram obtidos em uma única configuração de hardware e software, de modo que seus valores absolutos não devem ser considerados universais. Em entradas pequenas, medições da ordem de microssegundos estão mais sujeitas a variações do ambiente de execução. Em entradas muito grandes, a adoção de um limite de tempo pode produzir observações censuradas: um timeout permite concluir apenas que a execução não

terminou dentro do limite estabelecido, e não qual seria seu tempo total de resolução.

5. Conclusão

Os experimentos mostram que o aumento do tamanho do tabuleiro eleva o custo de resolução, mas também evidenciam que o desempenho depende tanto da estratégia adotada quanto das características de cada instância. O Rule-Based conseguiu reduzir de forma expressiva a quantidade de busca recursiva em diversos casos, diminuindo chamadas recursivas, guesses e backtracks. Entretanto, essa redução não se converteu necessariamente em menor tempo de execução, pois a aplicação das regras também possui custo, especialmente na procura por combinações associadas a naked tuples.

Os resultados também reforçam a importância de interpretar as medições em conjunto com a análise teórica. Em intervalos pequenos de entrada, o crescimento observado pode aparentar comportamento mais simples do que o pior caso previsto para os algoritmos. Assim, os gráficos experimentais não devem ser utilizados isoladamente para determinar a classe assintótica, mas como evidência complementar sobre o comportamento das implementações nas instâncias analisadas.

Dessa forma, a comparação evidencia um compromisso entre redução do espaço de busca e custo de processamento. O Backtracking apresenta uma estratégia mais simples, porém pode expandir uma árvore de busca combinatória. O Rule-Based pode reduzir significativamente essa busca por meio de deduções, mas adiciona o custo computacional das próprias regras e também recorre ao Backtracking quando elas são insuficientes. As limitações observadas nos experimentos e na generalização para tabuleiros maiores fazem, portanto, parte da análise de escalabilidade dos dois métodos.

6. Referências

BERGGREN, Patrik; NILSSON, David. A study of Sudoku Solving Algorithms. Trabalho de Conclusão de Curso. KTH Royal Institute of Technology, Stockholm, 2012.

OPENAI. ChatGPT. Ferramenta de inteligência artificial generativa. OpenAI, 2026. Disponível em: <https://chatgpt.com/>. Acesso em: 10 set. 2026.